Computer Graphics

第1回　ガイダンス

情報学部 情報学科 情報メディア専攻
清水 哲也

2026年度 後学期

今日の内容

1. コンピュータグラフィックスとは
2. この授業の目的
3. この授業で学ぶこと
4. 授業全体の流れ
5. Processingについて
6. 課題・試験・評価
7. 履修にあたって
8. 【重要】授業運営について

Computer Graphics

コンピュータグラフィックスとは何か？

Computer Graphicsとは

Computer Graphics（CG） とは，

> コンピュータによる計算を使って画像や映像を生成・表現する技術

です．

CGは非常に幅広い分野で利用されています．

- ゲーム
- 映画・アニメーション
- VR / AR / XR
- シミュレーション
- データ可視化
- デジタルアート

CGを作るためには？

CGでは，コンピュータの中で

「**図形をどのように表現するか**」

を考える必要があります．

例えば，

- 円をどこに描くか
- 物体をどの方向へ動かすか
- 3D空間で物体を回転させるにはどうするか
- 光が当たった物体をどう描画するか

といった処理を，**数値と計算によって記述**します．

CGを支えるもの

CGでは，主に次の知識を組み合わせます．

プログラミング

図形やアニメーションをコンピュータに計算させる

数学

位置，方向，回転，運動などを数式として表現する

この授業では

CG専用ソフトウェアを操作して映像を制作することが主目的ではありません.

例えば,

- Photoshopによる画像編集
- Illustratorによるイラスト制作
- Blenderによる3Dモデリング

を学ぶ授業ではありません.

この授業の考え方

「CGを使う」から「CGを理解する」へ

CGソフトを利用すると，「何ができるか」は分かります．

この授業ではさらに，

「コンピュータの中で何が計算されているのか」

をプログラミングを通して理解します．

この授業の目的

授業の目的

Processingによるプログラミングを通して，

「コンピュータグラフィックスの基礎概念と基本的なアルゴリズムを理解すること」

を目的とします．

Processingの操作方法を覚えること自体が この授業の目的ではありません．

この授業で目指すこと

授業終了時には，

1. **CGの基本概念を説明できる**
2. **CGの基本的な処理をプログラムとして実装できる**
3. **複数の技法を組み合わせて表現を設計できる**

ことを目指します．

到達目標

この授業では，主に次の能力を身につけます．

- 2D・3D CGの基本的な仕組みを理解する
- 図形を使ったアニメーションを実装する
- 座標変換を利用して図形を操作する
- 3Dモデルとアニメーションの関係を理解する
- 物理モデルを使った運動を実装する
- CG技術を組み合わせて作品を制作する

この授業で学ぶこと

1. 図形を描く

まずは，CGの最も基本となる「図形を描く」ところから始めます．

- 座標
- 点
- 直線
- 四角形
- 円
- 色

Processingを使いながら，2次元CGの基本的な考え方を確認します．

2. プログラムで図形を生成する

プログラミングを利用すると，同じ図形を大量に描いたり，
規則にしたがって配置したりできます．

扱う内容

- 変数
- 繰り返し
- 条件分岐
- 乱数

単純な図形から複雑なパターンを生成します．

3. アニメーション

静止した図形から，

時間によって変化するCG

へ発展します．

- `setup()`
- `draw()`
- フレーム
- フレームレート
- 位置の更新

CGに「時間」という概念を加えます．

4. 座標変換

図形そのものを変更するのではなく，

図形を描く座標系を変換する

という考え方を学びます．

- 平行移動
- 回転
- 拡大・縮小

5. 3D Computer Graphics

2次元から3次元へ拡張します.

$$(x, y) \to (x, y, z)$$

扱う内容

- 3次元座標
- X・Y・Z軸
- 3次元回転
- 3Dモデル
- 透視的な表現

6. 光と陰影

3D CGでは,

物体の形だけでなく「光」も重要

です.

光と物体の関係によって,

- 明るさ
- 陰影
- 奥行き感
- 立体感

が生まれます.

7. 物理モデルとアニメーション

より自然な動きを作るために，物理モデルを利用します．

例えば，

- 速度，加速度
- 重力，摩擦力
- 衝突
- 放物運動

などです．

位置・速度・加速度を
ベクトルとして扱います．

8. 数学とCG

数学を使うことで，さまざまな視覚表現を生成できます．

この授業では，

- 三角関数
- 極座標
- 再帰
- フラクタル
- 螺旋
- 数理造形

などをCG表現へ応用します．

Generative Art

プログラムに

- 数学的規則
- 乱数
- 反復
- 再帰

などを与えることで,

プログラム自身が視覚表現を生成できます.

このような表現は「**Generative Art**」とも呼ばれます.

授業全体の流れ

授業計画

回	日付	内容
1	2026-09-25	ガイダンス
2	2026-10-02	Processingの基礎と図形
3	2026-10-09	繰り返し・条件分岐
4	2026-10-16	アニメーション
5	2026-10-23	座標変換
6	2026-11-06	3Dモデル
7	2026-11-13	陰影・証明
8	2026-11-20	中間試験

授業計画

回	日付	内容
9	2026-11-27	物理モデルに基づくアニメーション I
10	2026-12-04	物理モデルに基づくアニメーション II
11	2026-12-11	数学アート I
12	2026-12-18	数学アート II
13–15	2027-01-08 ～ 22	オリジナル作品制作
16	2027-01-29	作品総評・意見交換

授業の流れ

1. 解説

CGの概念やアルゴリズムを理解する

↓

2. Processingによる実装

実際にプログラムを書く

↓

3. 演習・課題

自分でプログラムを変更・発展させる

「見る」だけでは分からない

プログラムは，「**実際に書いて，実行して，変更する**」ことで理解できます．

授業では，

```
読む
  ↓
実行する
  ↓
変更する
  ↓
結果を確認する
```

というサイクルを繰り返します．

Processingについて

Processing

この授業では Processing を使用します.

Processingは,

- Javaをベースとしたプログラミング環境
- CGやインタラクティブ表現に適している
- 少ないコードでグラフィックスを扱える

という特徴があります.

なぜProcessingを使うのか

Processingは，

「CGの基本的なアルゴリズムを

比較的シンプルなコードで確認できる」

ためです．

重要なのは，

> なぜその処理でCGが描画できるのか

を理解することです．

課題・試験・評価

評価方法

シラバスでは，次の割合で評価します．

評価項目	割合
受講態度	10 %
授業中の活動	10 %
予習・復習	30 %
成果物・発表	20 %
試験	30 %

課題

授業では，理解を定着させるために課題を実施します．

課題では，

「授業で扱ったプログラムをそのまま提出するのではなく，
理解した内容を使って自分で実装すること」

が重要です．

課題の提出方法・期限については，各回の授業およびMoodleで案内します．

フィードバック

提出された課題については，

- 次回授業での解説
- Moodleによる評価結果の返却

などを通してフィードバックします．

間違えた箇所や理解できなかった箇所は，そのままにせず確認してください．

中間試験

授業前半の理解度を確認するため，「中間試験」を実施します．

主に，

- CGの基本概念
- Processingによる実装
- アニメーション
- 座標変換
- 3D CG

などについて確認します．

オリジナル作品制作

授業後半では，

それまでに学習した技術を組み合わせて
オリジナル作品を制作します．

重要なのは，

高度な作品を作ることだけではなく，
授業で学んだCG技術を適切に利用すること

です．

履修にあたって

必要なプログラミング知識

この授業では，Processingを使用するため，
基本的なプログラミング経験があることを前提とします．

特に，

- 変数
- `if`
- `for`
- 関数
- 配列

などを利用します．

Javaプログラミングを履修していることが望ましいです．

数学について

CGでは数学を利用しますが,

高度な数学を最初からすべて理解している必要はありません.

授業の中で必要になったものを確認しながら使います.

主に,

- 座標
- 三角関数
- ベクトル
- 行列
- 極座標

などが登場します.

数学は「計算問題」ではない

この授業で数学を使う目的は，数式を解くことではありません．

数学を

「図形や動きを表現するための道具」

として使います．

例えば，

$$x = r\cos\theta$$
$$y = r\sin\theta$$

と書けば，

円周上の位置を計算できます．

分からないときは

プログラムは，

1 文字違うだけでも動かない

ことがあります．

エラーが出たときは，

1. エラーメッセージを読む
2. 該当行を確認する
3. 直前に変更した部分を確認する
4. 小さな単位で実行する

という手順で原因を探してください．

授業で大切にしてほしいこと

「動いた」で終わらない

プログラムが動いたら，

- なぜ動いたのか
- 値を変えたらどうなるか
- 別の図形にしたらどうなるか
- 別のアルゴリズムに応用できないか

を考えてください．

そこからCGの理解が始まります．

この授業のゴール

16回の授業を通して，

図形
↓
アニメーション
↓
座標変換
↓
3D
↓
光・陰影
↓
物理
↓
数学
↓
オリジナル表現

CGを

「見る」側から

「作る」側へ

【重要】授業運営について

授業運営について

履修者数によって，授業の運営方法が変わります．

履修者数	運営方法	担当
120名未満	1クラス運用	清水
120名以上	2クラス運用	山﨑先生・清水

基本方針

履修者数が **120名未満の場合は1クラス** で授業を実施します．

履修者数が **120名以上の場合は2クラス** に分けて授業を実施します．

2クラス運用となる場合

履修者数が **120名以上** となった場合，

- **山崎先生クラス**
- **清水クラス**

の2クラスで授業を実施します．

連絡時期

2クラス運用となる場合は，「**第2回授業（10月2日（金））の9時まで**」にアナウンスします．

授業に関するアナウンスを必ず確認してください．

次回

Processingの基本と図形描画

- Processingの基本操作
- 座標系
- 基本図形
- RGBカラー

実際にプログラムを書いてCGを描画してみます.